CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G2
SYSTEMS INTEGRATION AND ARCHITECTURE 1

FUNCTIONAL REQUIREMENTS SPECIFICATION (FRS)

Project Title: Mini App – User Registration & Authentication

Prepared By: Catamco, Danise Bianca B.

Date of Submission: February 8, 2026

Version: 1

Table of Contents

1. Introduction....3
   1.1. Purpose....3
   1.2. Scope....3
   1.3. Definitions, Acronyms, and Abbreviations....3
2. Overall Description....3
   2.1. System Perspective....3
   2.2. User Classes and Characteristics....3
   2.3. Operating Environment....3
   2.4. Assumptions and Dependencies....3
3. System Features and Functional Requirements....3
   3.1. Feature 1:....3
   3.2. Feature 2:....3
4. Non-Functional Requirements....3
5. System Models (Diagrams)....4
   5.1. ERD....4
   5.2. Use Case Diagram....4
   5.3. Activity Diagram....4
   5.4. Class Diagram....4
   5.5. Sequence Diagram....4
6. Appendices....4

1. Introduction

1.1. Purpose

- This system is a mini application that allows users to register, log in, and log out. It ensures that only authenticated users can access protected pages, and that user data is stored securely. The intended audience for this document includes developers, testers, and stakeholders who need a clear understanding of the system functionality.

1.2. Scope

The system provides the following functionality:

- User registration
- User login and authentication
- User logout
- Access control for protected pages

The system does not include additional features like password recovery, email verification, or advanced dashboards. Its main focus is on providing a simple and secure authentication process.

1.3. Definitions, Acronyms, and Abbreviations

- JWT – JSON Web Token, used for user authentication
- UI – User Interface (React frontend)
- API – Application Programming Interface (Spring Boot backend)
- ERD – Entity Relationship Diagram
- DTO – Data Transfer Object

2. Overall Description

2.1. System Perspective

The mini app consists of three main components:

- Frontend (React UI): Provides forms and buttons for user actions.
- Backend (Spring Boot API): Handles business logic, authentication, and database interaction.
- Database: Stores user data securely.

2.2. User Classes and Characteristics

- Guest User: Can register and log in; cannot access protected pages.
- Authenticated User: Can log in and log out; has access to protected content.

2.3. Operating Environment

- Frontend: React JS, running in a browser
- Backend: Spring Boot

- Database: MySQL
- Tools: Draw.io

2.4. Assumptions and Dependencies

- Users must provide a valid email to register.
- The system relies on a stable connection between frontend, backend, and database.
- JWT is used for authentication.
- Passwords are securely hashed before storing in the database.

3. System Features and Functional Requirements

Describe each major feature of the system and its functional requirements.

3.1. Feature 1:User Registration

Description: Allows a new user to create an account using first name, last name, email, and password.
Functional Requirements:
- The system checks if the email is already registered.
- Passwords are hashed before saving to the database.
- Successful registration returns a confirmation message.

3.2. Feature 2: User Login

Description: Allows registered users to log in using email and password.
Functional Requirements:
- The system validates the email and password.
- Successful login returns a JWT token for session management.
- Invalid credentials return an error message.

3.3. Feature 2: User Logout

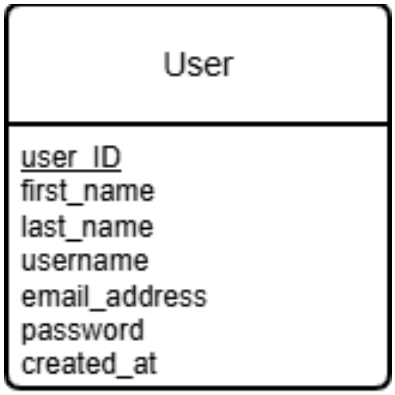
Description: Allows authenticated users to log out securely.
Functional Requirements:
- The system invalidates the JWT token.
- After logout, protected pages cannot be accessed.

4. Non-Functional Requirements

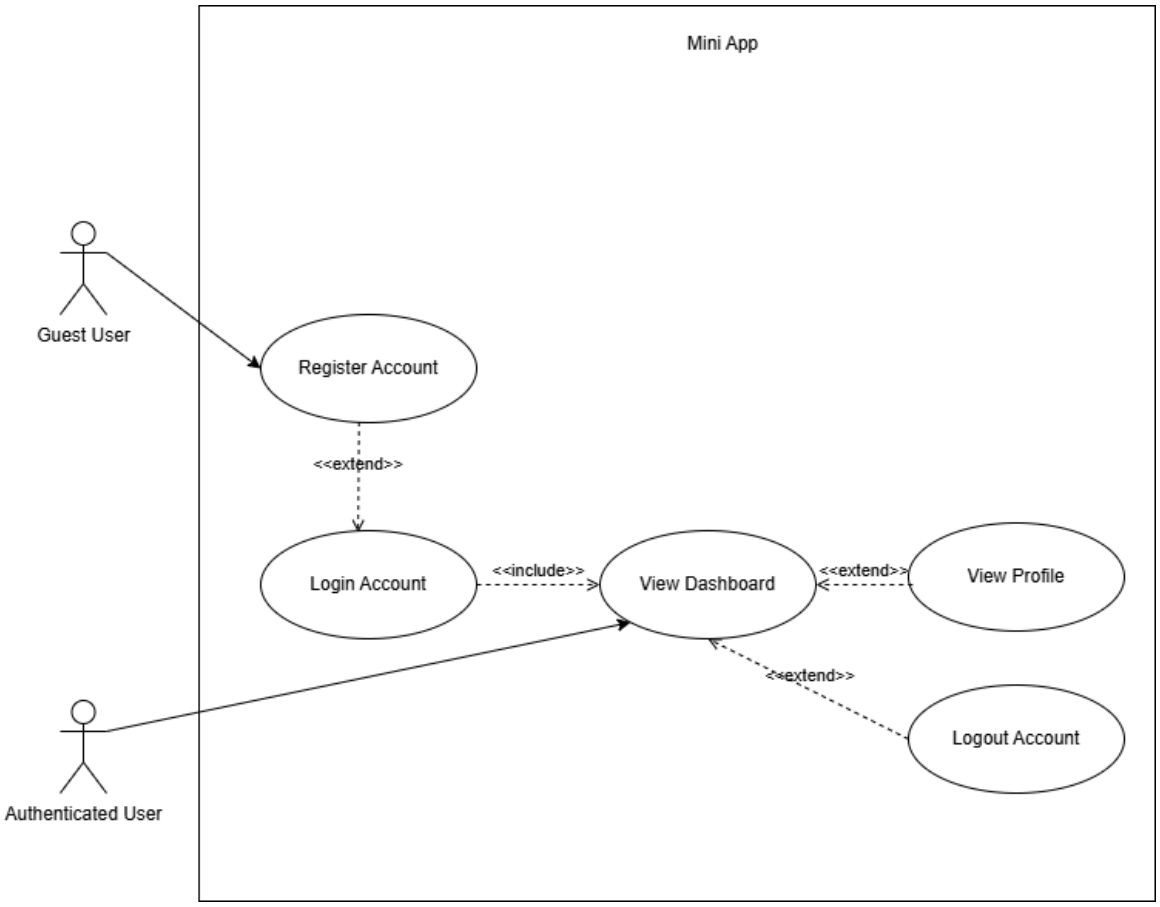
- **Performance**: API responses should be under 5 seconds.
- **Security:** Passwords are hashed; JWT tokens expire after 24 hours.
- **Usability:** Simple and intuitive interface for registration, login, and logout.
- **Reliability:** Handles invalid inputs gracefully.
- **Maintainability:** Modular backend design allows for future enhancements.

5. System Models (Diagrams)

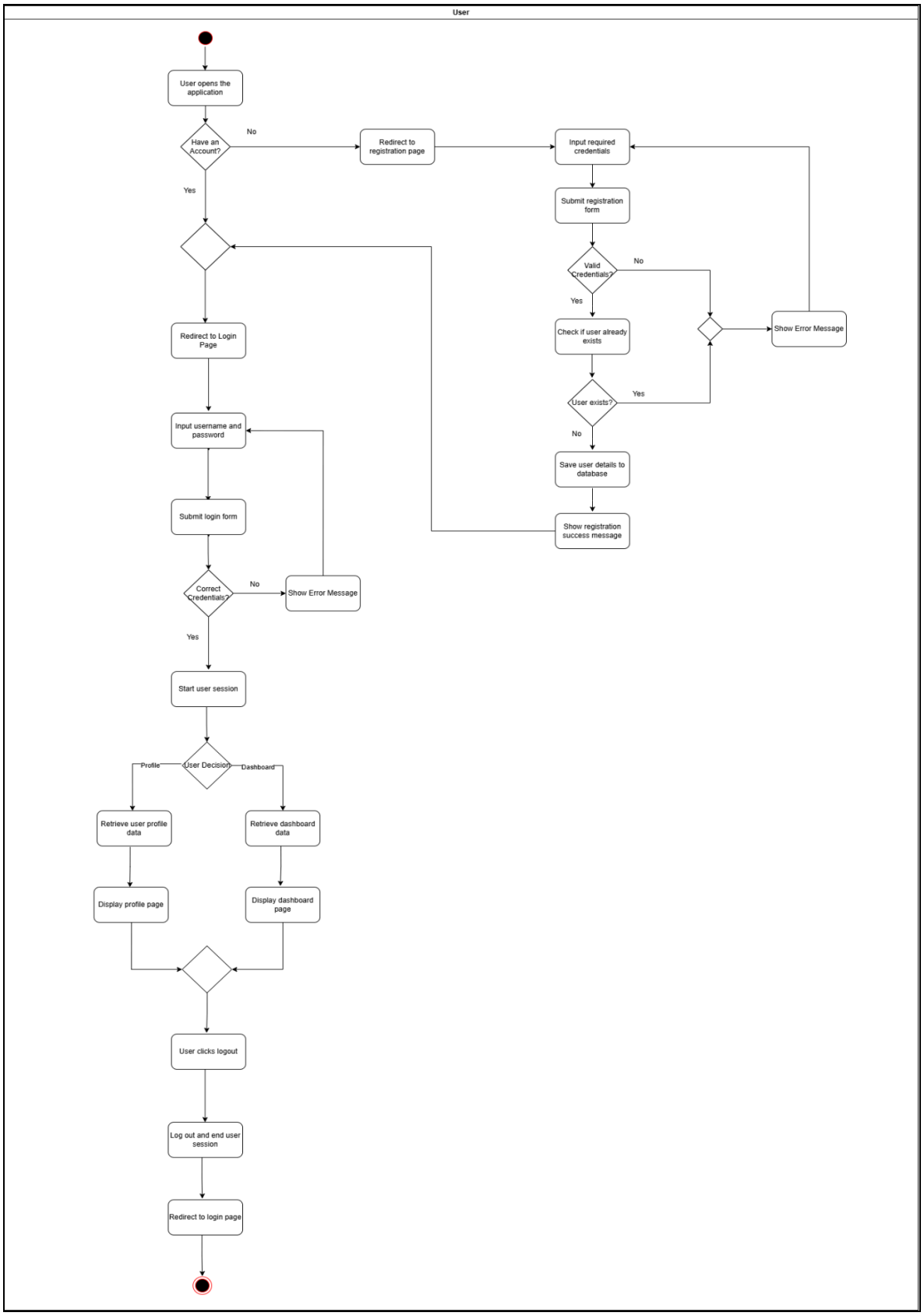
5.1. ERD



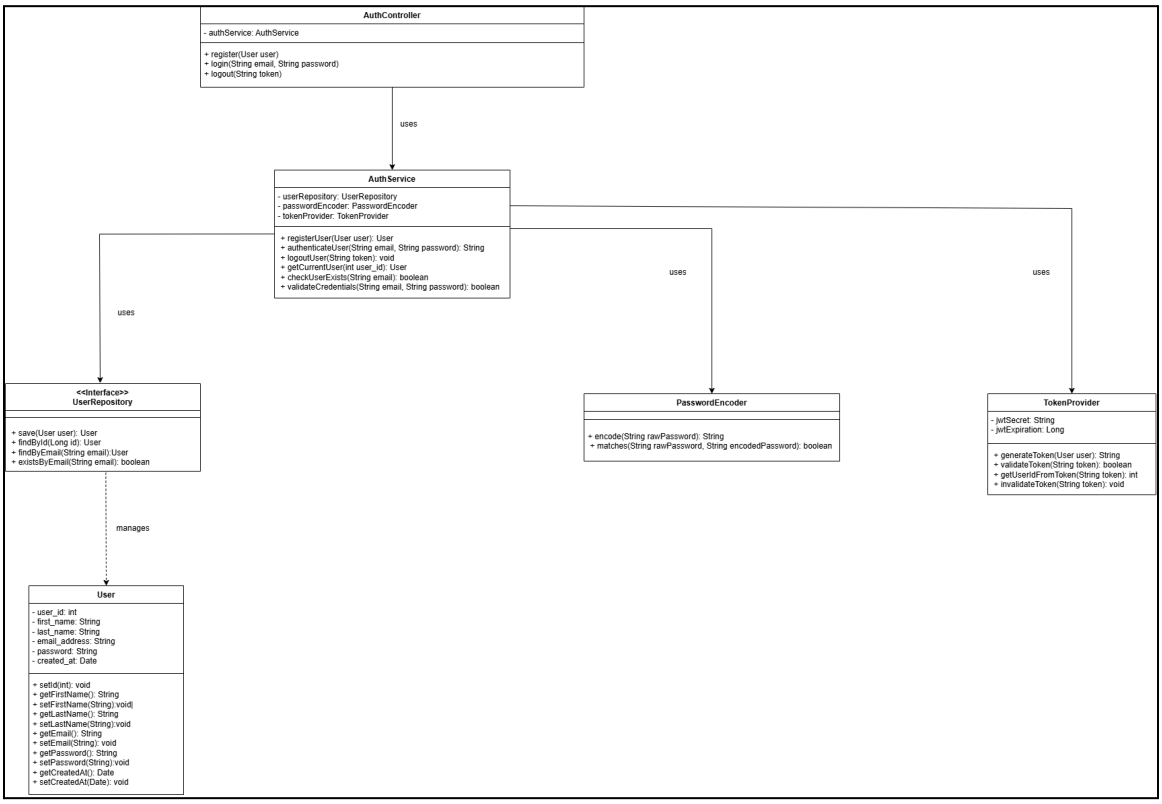
5.2. Use Case Diagram



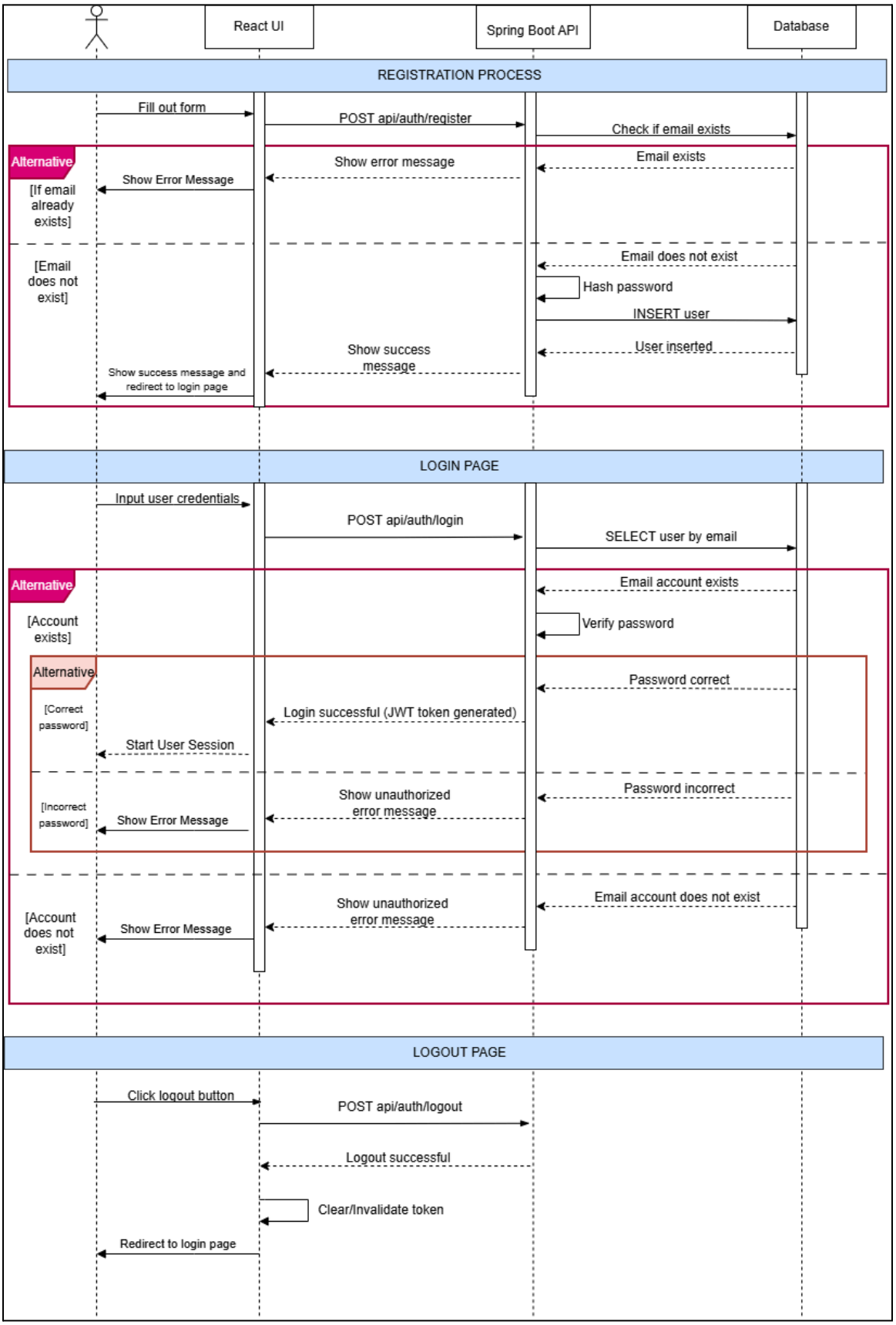
5.3. Activity Diagram



5.4. Class Diagram



5.5. Sequence Diagram



6. Appendices

Tools used: Draw.io, Spring Boot, React, MySQL/H2

References:

- Visual Paradigm – What is an Activity Diagram? https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-activity-diagram/
- diagrams.net – Draw.io, online diagram tool https://www.diagrams.net/
- Visual Paradigm – What is a Sequence Diagram? https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-sequence-diagram/